

Homework 2

Total Marks: 40**Deadline:** Mar 29, 2026, at 11:55 pm

General Instructions: You may partner with up to 3 other students (a group of up to 4) to submit a single write-up in a single group. We encourage discussion with other students, even if they are not in your group. Each student must develop a solid understanding of all the questions, even when discussing and collaborating with others. Merely dividing the problems among group members and putting those together before submission will severely impact your learning, and we **strongly advise against it**. Furthermore, while it is okay to use online resources, including **Generative AI** such as **ChatGPT**, for learning purposes, **blatant copying** or **mindlessly querying** the Generative AI is **forbidden**. If there are any confusions or questions, post those on the WhatsApp group or see your TA or Instructor.

Submission Instructions: Typed or hand-written solutions (only in PDF format) can be submitted on LMS before the deadline. Email submissions WILL NOT be accepted. *Only one submission per group is needed; the first page must clearly mark the names and student IDs of all group members.*

Question 1 [5 marks] In Bitcoin, each user's balance is represented using the hash of a public key. A user's name or identity is not stored on the blockchain. An adversary may still use the transaction history associated with a public key in order to arrive at the identity of a user. Give an example of how this is possible and devise two strategies that may be used to protect the anonymity of a user.

Solution: A user's identity may be identified by correlating the activities of a user on the blockchain with people in real life. For example, I bought a Tesla for exactly 5.24234232 BTC. A third party observes that I own a Tesla now and may look up the price of a Tesla (in BTC). Then they can check for recent transactions in which exactly 5.24234232 BTC were traded and guess my identity on the blockchain. This is not a perfect approach, but it can work with a high degree of success in most circumstances.

Many strategies may be used for this purpose. The most common (and recommended) strategy is to lock each UTXO to a *new* public/private key pair. (In fact, this is the most recommended approach if you are trying to future-proof the BTC blockchain.

Another strategy is to involve multi-input and multi-output transactions. If multiple people are sending things individually, it is easier to track who is sending to whom. If a bunch of exchanges happen as a part of one transaction, this is a lot more difficult.

Further obfuscation is also possible, such as sending the same amount to public keys belonging to the same individual before sending them to the third party, and so on.

Question 2 [6 marks]

- a) Explain why strictly more than half of the network resource guarantees eventual chain overtake.
- b) Assume the network consensus currently considers Block#271 to be the latest block. A malicious miner controlling 51% of the network has the ability to create a private fork. Describe a double-spend attack that this miner could use against an exchange that credits deposits only after k confirmations (the exchange deposits cash into the user's

account in exchange for cryptocurrency after k blocks have been mined after it). Be explicit about the miner's actions and the point at which the exchange would be tricked.

c) Give reasons why 51% attacks are still rare in practice.

Solution: (a) With more than 50% of the resource, the attacker's private chain grows faster on average than the public chain. This is guaranteed by the fact that more hashing power implies faster mining. Given enough time, the private chain will eventually overtake the honest chain.

(b) The attacker first sends a deposit transaction to the exchange in Block#272 and allows it to be included in the public blockchain so that everyone sees it. While the honest network continues to build on top of that block, the attacker secretly starts mining their own private fork beginning from Block#272, but in this block, he does not include the transaction in which he sends the deposit.

As the honest chain grows, the attacker keeps their fork hidden but continues to mine on it using their majority hash power. Because they have more than 50% of the network's compute power, their private fork will grow faster than the public chain on average.

Once the exchange sees k confirmations on the public chain, it considers the deposit final and credits the attacker's account, allowing them to withdraw or trade the assets. After successfully moving the funds, the attacker releases their longer private fork to the network. According to the longest-chain rule, all honest nodes switch to the attacker's fork, which does not include the original deposit transaction. So consequently, the malicious miner gets away with 'tampering' the blockchain, in a sense.

(c) 51% attacks are rare because they are very expensive to sustain, they devalue the attacker's own holdings by reducing trust in the currency, exchanges may pause or raise confirmation requirements during an attack, honest miners may coordinate a software update or checkpoint to override or undermine the attack, and large mining pools risk losing reputation and users if they are caught attempting such attacks.

Question 3 [5 marks] Imagine that a blockchain is about to activate a soft fork that reduces the block size limit. Most miners have upgraded, but Aqib controls 51% of the network's compute power and refuses to upgrade.

- a) Describe what will happen to the chain if Aqib mines blocks that violate the new, smaller block size limit. Will the chain split temporarily, permanently, or not split at all?
- b) Describe what will happen in the other scenario where Aqib, with 51% of compute power upgrades, while all the other nodes stay non-updated. Will the chain split temporarily, permanently, or not split at all? Will he be able to double-spend there?

Solution: (a) Aqib's blocks exceed the new (smaller) size limit, so *upgraded* nodes will reject them as invalid and continue building on the last block that obeys the new rule. Because Aqib controls 51% of hash power and keeps mining *old-rule* (large) blocks, his chain will grow faster among *non-updated* nodes, while upgraded nodes extend a separate chain. This produces a **persistent (effectively permanent) split** as long as Aqib continues to mine invalid-under-new-rules blocks with majority hash power. The upgraded minority cannot "catch up" on Aqib's chain because they refuse those blocks by rule; likewise, non-updated nodes will prefer Aqib's longer chain. The split would only end if

Aqib stops or upgrades.

(b) Now Aqib *does* upgrade and still has 51% hash power, while the rest remain non-updated. In a soft fork, upgraded blocks are *stricter* but still look valid to old nodes, so old nodes will accept Aqib's smaller blocks. With his majority producing most blocks under the new rule, the network **does not split**: both upgraded and non-updated nodes converge on the same chain (the upgraded rule chain). However, because Aqib has the majority of the hash power, he can still **double-spend** against parties relying on a fixed k confirmations.

Question 4 [5 marks] When a soft fork is invoked, does it need the majority of nodes to upgrade to ensure that a single chain emerges? Why or why not?

Solution: Yes, in practice, a majority of the network's hash power must upgrade for a soft fork to cleanly converge to a single chain.

A soft fork introduces stricter rules (for example, a smaller block size or a new transaction format). Upgraded nodes will reject any block that breaks the new rules, while non-updated nodes will still accept them because they are valid under the old rules. This means that if many non-updated miners keep producing "old-rule" blocks, two chains can temporarily exist:

- One chain that follows the new, stricter rules (grown by upgraded miners).
- Another chain that includes blocks that violate the new rules (grown by non-updated miners).

If the majority of hash power has upgraded, the chain with valid upgraded blocks will grow faster on average. Because of the longest-chain rule, even non-updated nodes will eventually follow this chain since all its blocks are still valid under their old rules. This causes the "old-rule" chain to be abandoned (its blocks become stale), and the entire network reconverges on a single chain.

If a majority does *not* upgrade, the reverse happens: the upgraded chain grows more slowly and risks being permanently left behind, leading to a lasting split where upgraded nodes never accept the majority's blocks. Therefore, the majority upgrade is necessary to guarantee a single, unified blockchain history.

Question 5 [5 marks] Read up on "Grover's algorithm". Will the existence of this algorithm (and similar algorithms) affect cryptographic hash functions such as SHA256? What about symmetric key encryption, such as AES-128?

Solution: Grover's algorithm reduces brute force complexity from 2^{256} to 2^{128} for SHA256, still infeasible. AES-128 would go from 2^{128} to 2^{64} operations, still beyond reach for near-term quantum computers.

Question 6 [6 marks]

- a) In simple words, explain what a soft fork and a hard fork are, focusing on what happens to nodes that do not upgrade.
- b) Give one example of a rule change for each type of fork (other than block size or mining reward) and explain why it would be a soft or a hard fork.

Solution: (a) A soft fork is when the rules of the blockchain get stricter, so upgraded nodes reject some blocks that older nodes would have accepted. Because the new rules are a

stricter version of the old rules, old nodes still see upgraded blocks as valid and stay on the same chain. A hard fork is when the rules change in a way that is not backward compatible; old nodes will see some new blocks as invalid and will stay on a separate chain, causing a permanent split if they do not upgrade.

(b) Example of a soft fork: adding a rule that all transactions must include a time lock before they can be spent. Old nodes still see such transactions as valid, so they follow the upgraded chain. Example of a hard fork: allowing a completely new type of transaction, such as one that supports a new kind of digital signature that old nodes cannot verify. Old nodes would reject these blocks because they do not understand the new transaction type, leading to a chain split.

Question 7 [4 marks] Suppose Aqib is a malicious miner who controls 51% of the network's compute power. This means he has complete control of the blocks he mines. As we have seen previously, he maliciously uses this power to double-spend his cryptocurrency. Is there any other way he can exploit his control over the mined blocks? Can he change the outputs of the fair transactions to his own address?

Solution: Even with 51% of the compute power, Aqib **cannot** make a block that violates consensus rules, such as spending non-existent UTXOs or changing transaction outputs. This is ensured by digital signatures and the UTXO database maintained by every user. If he did, the fair nodes would consider his blocks invalid and wouldn't append to his chain, regardless of it being the longest.

He, however, can change the coinbase transactions in all the blocks that he is privately mining to send the mining reward to his own address. In this way, the longer his chain, the more coins he can just "mint" for himself without violating any consensus rules.

Question 8 [4 marks] Explain the concept of non-repudiation in digital signatures. Why is this property critical for cryptocurrencies like Bitcoin? How does Bitcoin ensure it?

Solution: Non-repudiation is the property that prevents a sender from denying that they created and authorized a particular message or transaction.

This property is critical for cryptocurrencies like Bitcoin because transactions are irreversible; once recorded on the blockchain, they must be verifiable as genuinely authorized by the owner of the coins. Without non-repudiation, a user could later deny making a payment and dispute its validity.

Bitcoin ensures non-repudiation using **digital signatures** based on the sender's private key. Anyone can check that the signature matches the public key and the transaction data, proving that only the private key holder could have authorized the transfer.